Software Requirements Specification (SRS)
for
Project Argus: an AI-based Fraud Detection System

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the complete requirements for Project ARGUS. It describes the external behavior, functional requirements, and non-functional constraints of the AI-driven fraud detection engine designed specifically for the Indian payment ecosystem. This document serves as the primary reference for developers, data scientists, QA testers, and stakeholder.

1.2. Scope

ARGUS is a real-time, pre-authorization fraud detection middleware. It sits between a payment gateway and a core banking ledger. The system uses a machine-learning ensemble (XGBoost, LightGBM, Isolation Forest) combined with dynamic user behavioral profiling and graph-based network analysis to approve or block transactions in under 20 milliseconds. It is designed to combat modern financial threats including UPI scams (Digital Arrest, QR phishing), account takeovers, and organized mule account rings.

1.3. Definitions, Acronyms and Abbreviations

- **AI / ML:** Artificial Intelligence / Machine Learning.
- **API:** Application Programming Interface.
- **CSRF:** Cross-Site Request Forgery.
- **GIL:** Global Interpreter Lock (Python concurrency limitation).
- **NPCI:** National Payments Corporation of India.
- **RBI:** Reserve Bank of India.
- **SMOTE:** Synthetic Minority Over-sampling Technique.
- **TPS:** Transactions Per Second.
- **UPI:** Unified Payments Interface.
- **XAI:** Explainable Artificial Intelligence.

1.4. References

a. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
b. NPCI UPI Security Guidelines.
c. RBI Master Direction on Digital Payment Security Controls.

1.5. Overview

The remainder of this document is organized into four main sections. Section II provides a general description of the system's operating environment and constraints. Section III details the external interfaces. Section IV exhaustively lists the functional system features. Finally, Section V defines the non-functional requirements governing performance and security.

2. General Description

2.1. Product Perspective

ARGUS is not a standalone banking application; it is an intelligent middleware layer. It receives raw JSON transaction payloads from a bank's existing payment gateway, evaluates the risk using trained ML models, and returns a binary block/approve decision along with an Explainable AI (XAI) rationale. It also includes a standalone React-based Command Center dashboard for human fraud analysts.

2.2. Product Functions

- **Real-time Risk Scoring:** Evaluates transactions against a weighted ensemble of ML models.
- **Dynamic Behavioral Analysis:** Tracks user-specific spending patterns to calculate real-time Z-Scores.
- **Graph Network Analysis:** Detects cyclic money laundering loops and mule account clusters.
- **Cybersecurity Defense:** Prevents session hijacking, malicious referrer navigation, and automated bot inputs.
- **Case Management:** Provides a queue system for human analysts to manually review "High Risk" transactions.

2.3. User Classes and Characteristics

2.3.1. **Bank Fraud Analysts:** Primary users of the ARGUS Dashboard. They possess high domain knowledge of banking but low ML technical knowledge. They require clear, human-readable XAI explanations.

2.3.2. **System Administrators / Data Scientists:** Technical users responsible for retraining the ML models offline and updating system thresholds via configuration files.

2.3.3. **End Customers:** Invisible users who do not interact with ARGUS directly but are impacted by its latency and false-positive decline rates.

2.4. Operating Environment

- **Backend:** Python 3.11+, FastAPI framework.
- **Machine Learning:** Scikit-Learn, XGBoost, LightGBM, NetworkX.
- **Database:** SQLite (prototype) / PostgreSQL & Redis (production).
- **Frontend:** React 18, Vite, Tailwind CSS.
- **Deployment:** Linux-based server environments capable of asynchronous I/O.

2.5. Design and Implementation Constraints

- **Latency Constraint:** The core inference engine MUST return a decision in <20 milliseconds to prevent payment gateway timeouts.
- **Concurrency Constraint:** Because Python relies on the GIL, horizontal scaling and external inference servers (like NVIDIA Triton) must be planned for production loads exceeding 10,000 TPS.
- **Data Availability:** The graph detection module requires the entire network topology to reside in memory, constraining the maximum number of simultaneous nodes before requiring a dedicated graph database like Neo4j.

3. External Interface Requirement

3.1. User Interfaces

- The system shall provide a React-based web dashboard.
- The dashboard shall utilize a strict CSS Grid layout for enterprise-grade alignment.
- The dashboard must feature a "Live Traffic" view utilizing WebSockets to stream incoming transactions without manual page refreshes.

3.2. Hardware Interfaces

- The backend requires multi-core CPU architectures optimized for high-speed mathematical matrix multiplication (required by XGBoost/LightGBM).

3.3. Software Interfaces

- **Payment Gateway Interface:** The system must expose a REST API endpoint (POST /api/v1/analyze) that accepts standard JSON transaction payloads.
- **Notification Interface:** The system shall interface with standard SMTP/SMS gateways to dispatch automated alerts for blocked transactions.

3.4. Communication Interfaces

- All client-server communication must occur over TLS 1.2+ (HTTPS/WSS).
- Internal communication between the FastAPI server and the React frontend will utilize WebSockets for sub-millisecond telemetry updates.

4. System Features (Functional Requirements)

4.1. Real-Time Machine Learning Inference

The core decision engine that evaluates incoming transactions.

FR-1.1: The system shall load pre-trained XGBoost, LightGBM, and Isolation Forest models into memory upon server startup.

FR-1.2: If pre-trained models are missing, the system shall bootstrap a lightweight dummy model to prevent server crashes.

FR-1.3: The system shall extract 34 numerical features from the raw JSON payload dynamically.

FR-1.4: The system shall calculate a weighted final risk score (e.g., 30% XGBoost, 25% LightGBM, 15% Isolation Forest).

4.2. Dynamic User Behavior Profiling

A statistical layer that personalizes risk thresholds per user.

FR-2.1: The system shall maintain a rolling history of the last 100 transactions for active users.

FR-2.2: The system shall dynamically calculate the mean and standard deviation for each user.

FR-2.3: The system shall calculate a Z-score for incoming transactions to determine statistical deviation from the user's norm.

FR-2.4: The system shall periodically persist these profiles to the disk to survive server restarts.

4.3. Fraud Ring and Graph Detection

Analyzes relationships between sender and receiver accounts.

FR-3.1: The system shall maintain a directed graph of transactions.

FR-3.2: The system shall detect cyclic money transfer patterns (e.g., A -> B -> C -> A).

FR-3.3: The system shall flag accounts with abnormally high in-degree and out-degree ratios as potential Mule Accounts.

4.4. Cybersecurity & Phishing Protection

Defends against device-level and session-level attacks.

FR-4.1: The system shall inspect HTTP Referrer headers against a blacklist of known free TLDs (e.g., .tk, .ml).

FR-4.2: The system shall parse Referrer domains for typosquatting keywords (e.g., "bank-secure").

FR-4.3: The system shall analyze device telemetry (mouse movements, time-on-page) to detect automated bot submissions.

4.5. Case Management and Analyst Workflows

The human-in-the-loop audit system.

FR-5.1: The system shall log all transactions to an immutable SQLite database ledger.

FR-5.2: The system shall automatically route transactions with a risk score > 0.35 to a manual review queue.

FR-5.3: Analysts shall be able to update the status of a case to APPROVED, REJECTED, or INVESTIGATING.

4.6. Explainable AI (XAI)

Translates opaque ML probabilities into human-readable rationale.

FR-6.1: The system shall map raw feature importance weights to heuristic string headlines.

FR-6.2: The XAI module must output specific reasons (e.g., "AMOUNT_SPIKE: 5.2x user's average") rather than generic block codes.

5. Other Non-Functional Requirements

5.1. Performance Requirements

PR-1: The pre-authorization engine must process and return a decision for a single transaction in less than 20 milliseconds.

PR-2: The system must sustain a throughput of at least 1,000 Transactions Per Second (TPS) in the prototype environment.

5.2. Security Requirements

SR-1: The system must not log or store plain-text passwords or full unmasked credit card numbers in the transactions table.

SR-2: The system must actively validate expected CSRF tokens for all state-changing frontend requests.

SR-3: The React dashboard must sanitize all database outputs to prevent Cross-Site Scripting (XSS) attacks.

5.3. Reliability and Availability

RA-1: The system shall achieve 99.9% uptime.

RA-2: In the event of a database lock (SQLite concurrency limit), the ML inference engine must fail-open (approve transaction) or fail-closed based on global RBI configuration, rather than dropping the connection.

5.4. Usability

UR-1: The dashboard must utilize high-contrast visual cues (Red/Yellow/Green) to denote risk severity, allowing analysts to triage cases in under 3 seconds per view.

5.5. Scalability

SC-1: The system architecture shall support horizontal scaling (adding more FastAPI worker nodes) once the in-memory state is migrated to a Redis cluster.

SC-2: The ML pipeline must be modular to allow the addition of new models (e.g., deep learning sequence models) without rewriting the core inference engine.

6. Future Enhancements

- **Migration to Distributed Systems:** Transitioning from SQLite to PostgreSQL (for the ledger) and Redis (for dynamic user profiles) to enable true horizontal scaling.
- **Graph Database Integration:** Migrating the in-memory networkx graph fraud detector to a dedicated Neo4j instance to handle billions of nodes globally.
- **Continuous Online Learning:** Implementing an MLOps pipeline (e.g., Apache Airflow) to automatically retrain the XGBoost models weekly to combat "Concept Drift" as scammers evolve tactics.
- **Hardware Acceleration:** Exporting the models to ONNX format and serving them via NVIDIA Triton to bypass Python's Global Interpreter Lock (GIL) and achieve sub-millisecond latency.

7. Appendix

7.1. Sample Fraud Scenarios Addressed

- **Digital Arrest / Coercion:** A user suddenly empties their entire life savings in 4 successive UPI transfers to a brand new payee.
- **Mule Account Rings:** Money flows in a cyclic pattern: Account A → Account B → Account C → Account A.
- **Phishing / Typosquatting:** A user initiates a transaction where the HTTP Referrer is a cheap fake domain like paytm-secure.tk.
- **Zero-Day Attacks:** A completely new fraud strategy that the supervised XGBoost model has never seen, but is caught as a mathematical outlier by the unsupervised Isolation Forest.

7.2. Tools and Technologies

- **Languages:** Python 3.11+, JavaScript (ES6+), HTML/CSS.
- **Backend Frameworks:** FastAPI, Uvicorn, WebSockets.
- **Machine Learning:** Scikit-Learn, XGBoost, LightGBM, NetworkX, Imbalanced-Learn (SMOTE).
- **Frontend Frameworks:** React 18, Vite, Tailwind CSS, Recharts, PyVis.
- **Databases:** SQLite (Prototype Phase).

8. Conclusion

This SRS document defines the complete requirements for developing and deploying the ARGUS AI/ML-based Fraud Detection System. It serves as a comprehensive foundation for system design, development, regulatory testing, and production deployment within the Indian financial ecosystem.